Neural Networks: Introduction

2026-04-17

Introduction

Neural networks are compositions of linear maps and non-linear activation functions. A feed-forward network with L layers computes $f(x) = g_L(W_L g_{L-1}(\cdots W_2 g_1(W_1 x + b_1)\cdots) + b_L)$ where W_l are weight matrices, b_l biases, and g_l activation functions. The Universal Approximation Theorem (Cybenko, 1989) shows that even a single hidden layer with enough width can approximate any continuous function. Modern deep learning stacks hundreds of such layers to learn powerful representations for images, text, audio, and tabular data with implicit feature engineering.

Prerequisites

A working understanding of linear algebra (matrix multiplication), gradient descent, the chain rule for backpropagation, and basic machine-learning concepts (loss functions, train-test evaluation).

Theory

Perceptron: $\hat{y} = \text{sign}(w^\top x + b)$. Linear decision boundary; provably cannot solve XOR.

Multi-layer perceptron (MLP): two or more layers of linear + non-linear transformations. Universal approximator with one hidden layer of sufficient width.

Activation functions: - **Sigmoid**: $\sigma(x) = 1/(1+e^{-x})$, smooth but vanishing gradients in deep nets. - **tanh**: zero-centred sigmoid; better than sigmoid for hidden layers but still suffers vanishing gradients. - **ReLU**: $\max(0, x)$, the modern default for hidden layers; faster training and avoids vanishing gradients in positive regions. - **Leaky ReLU**, **GELU**, **SiLU**: variants that address ReLU's "dead-neuron" problem.

Training: minimise loss (MSE for regression, cross-entropy for classification) by gradient descent (or modern variants Adam, AdamW). Backpropagation computes gradients efficiently via the chain rule.

Assumptions

Sufficient data relative to model capacity (overparameterised models on tiny datasets memorise rather than generalise); features are normalised; loss is differentiable almost everywhere.

R Implementation

```
library(nnet)

set.seed(2026)
n <- 400
x1 <- rnorm(n); x2 <- rnorm(n)
y  <- factor(ifelse(x1^2 + x2^2 > 1.5, "out", "in"))
df <- data.frame(x1 = scale(x1)[, 1], x2 = scale(x2)[, 1], y = y)

idx <- sample(n, n * 0.7)
nn <- nnet(y ~ ., data = df[idx, ],
           size = 5, decay = 1e-3, maxit = 200,
```

```
            trace = FALSE)

pred <- predict(nn, df[-idx, ], type = "class")
mean(pred == df[-idx, "y"])
```

Output & Results

`nnet()` fits a small MLP with one hidden layer of `size` neurons. The test accuracy on the concentric-rings task demonstrates the network's ability to learn a non-linear decision boundary that linear classifiers cannot represent.

Interpretation

A reporting sentence: "A 5-neuron single-hidden-layer MLP with weight decay 1e-3 trained for 200 iterations achieved 91 % test accuracy on the concentric-rings classification task; logistic regression on the same data managed only 52 %, confirming that the non-linear decision boundary requires non-linear modelling capacity." Always state architecture, regularisation, and a linear baseline.

Practical Tips

- Always standardise inputs; unscaled features with different units make training unstable and slow.
- Combine multiple regularisers: weight decay (L2), dropout, early stopping based on validation loss.
- Initialise weights carefully: Xavier (for tanh) or He (for ReLU) for deep networks; `nnet` uses small uniform initialisation that suffices for shallow nets.
- For tabular data, tree-based ensembles (XGBoost, LightGBM) usually outperform MLPs; neural networks dominate where structure matters (images, text, audio, time series).
- Modern R neural-network workflows use `torch` or `keras` rather than `nnet`; the latter is fine for shallow nets and pedagogy but lacks the GPU support and modern optimisers needed for production.
- For very small datasets, regularise heavily (high decay, small networks, dropout) or fall back to linear / tree models; deep nets need data to generalise.

R Packages Used

`nnet::nnet()` for shallow MLPs; `torch` and `luz` for modern deep learning with GPU support; `keras` and `tensorflow` as the alternative TensorFlow-based stack; `parsnip::mlp()` for tidymodels integration; `neuralnet` for an alternative implementation with visual graph rendering.

For Reviewers

What to look for in a paper using this method.

- **Common misapplications.**
- **Diagnostics that should be reported but often aren't.**
- **Red flags in tables and figures.**
- **What to verify.**
- **What an adequate Methods paragraph must contain.**

See also — labs in this chapter

- Cross-validation, nested CV, bootstrap .632+
- Regularisation: ridge, lasso, elastic net
- Trees, random forests, gradient boosting
- Interpretability and SHAP
- Tabular neural networks with torch
- Imaging and sequence models intro

- tidymodels pipelines
- FDR, knockoffs, replication crisis